

F

F → f

F → E

Stack

Input

Action

\$	acgfh\$	push Start S
\$S	acgfh\$	$S \rightarrow aBDh$
\$hDBa	acgfh\$	pop
\$hDB	cghf\$	$B \rightarrow cC$
\$hDCc	cghf\$	pop
\$hDC	ghf\$	$C \rightarrow E$
\$hD	ghf\$	$D \rightarrow EF$
\$hFE	ghf\$	$E \rightarrow g$
\$hFg	ghf\$	pop
\$hF	fh\$	$F \rightarrow S$
\$hf	fh\$	pop
\$h	h\$	pop
\$	\$	Accept.

* Bottom Up parsing

Shift Reduce

LR

LR(0)

SLR(1)

CLR(1)

LALR(1)

$$\begin{aligned}
 &\Rightarrow \underline{id} + id * id \\
 &= E + \underline{id} * id \\
 &= E + E * id \\
 &= \underline{E * id} \\
 &= \underline{E * E} \\
 &= \underline{E}
 \end{aligned}$$

underlined part is called handle.

Tree pruning

possible handles = $\{ id, E+E, E*E \}$

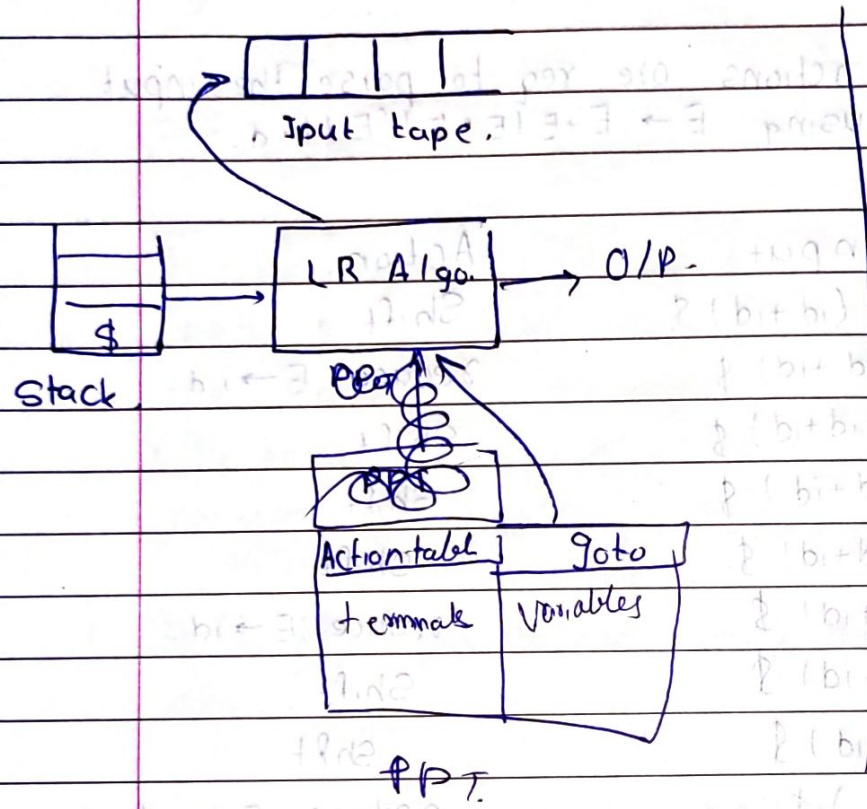
- Handle is a sentential form present on the right hand side such that a substring β that matches the RHS of a production $A \rightarrow \beta$
- The reduction of β to A is a state along the reverse of rightmost derivation

- 1 Shift Action: Parser shift the next token onto the stack.
- 2 Reduce Action: Parser reduces the RHS of a production to its LHS.
- 3 error: Parser discovered a syntax error.
4. Accept Action accept describe a successful parser.

Stack	Input	Action
\$	id + id * id \$	Shift
\$ id	+ id * id \$	Reduce $E \rightarrow id$
\$ E	+ id * id \$	Shift
\$ E +	id * id \$	Shift
\$ E + id	* id \$	Reduce $E \rightarrow id$
\$ E + E	* id \$	Reduce $E \rightarrow E + E$
\$ E	* id \$	Shift
\$ E *	id \$	Shift
\$ E * id	\$	Reduce $E \rightarrow id$

LR $\rightarrow$ Left to right Scanning
Reverse of rightmost derivation

LR(0) Closure ()
goto ()



We need to first identify LR(0) items.

$S \rightarrow ABC$

$\downarrow$
 $S \rightarrow \cdot ABC$

$S \rightarrow A \cdot BC$

$S \rightarrow AB \cdot C$

$S \rightarrow ABC \cdot$

} intermediate LR(0) items.
(reduce) / Handle is detected
Action

* Construction of PPT

1) Augment the given grammar.

$S \rightarrow AA \Rightarrow S' \rightarrow S$
 $A \rightarrow aAb$
 $S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

Draw back of shift reduce.

↳ detection of proper handle is not provided

2) Construct FSM (Finite State Machine)

$$M = (Q, \Sigma, q_0, F, \delta)$$

$$\Sigma = \{a, b, S', S, A\}$$

$$I_0 : S' \rightarrow \cdot S$$

I_0 - Initial LR(0) item set

(when variable is present right immediate to the dot apply closure operation)

$$I_0 : S' \rightarrow \cdot S$$

$$S \rightarrow \cdot AA$$

$$A \rightarrow \cdot aA$$

$$A \rightarrow \cdot b$$

$$I_2 : S \rightarrow A \cdot A$$

$$A \rightarrow \cdot aA$$

$$A \rightarrow \cdot b$$

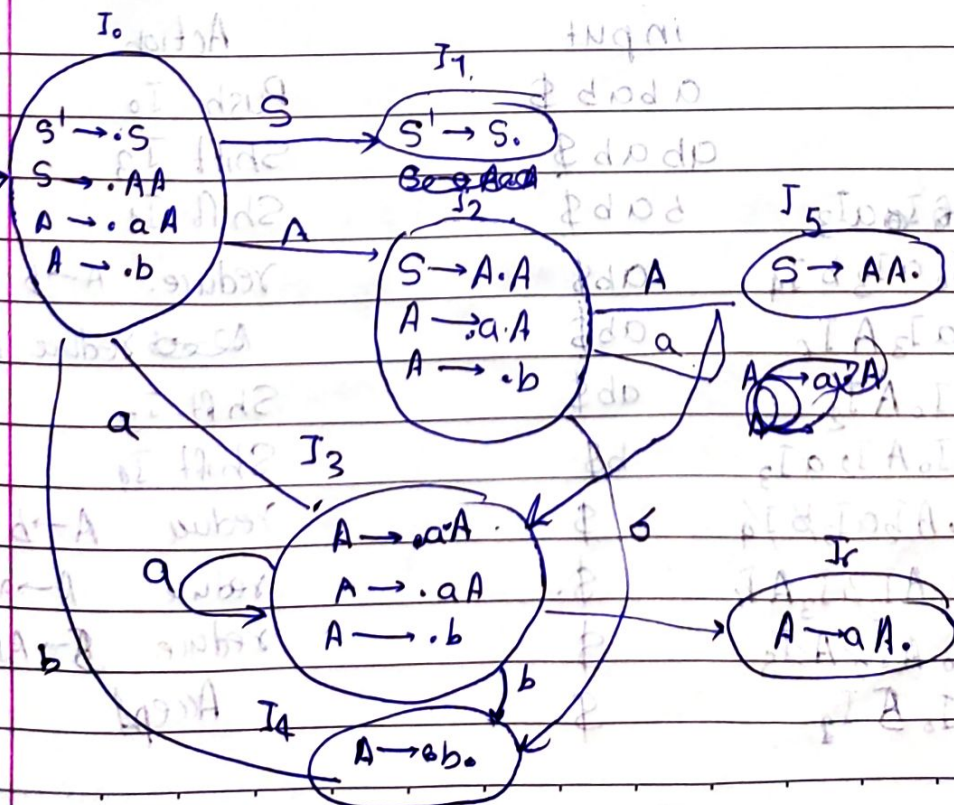
$$I_1 : S' \rightarrow S \cdot$$

$$S \rightarrow A \cdot A$$

$$A \rightarrow a \cdot A$$

$$A \rightarrow b \cdot$$

LR(0) : Items



	Action			Goto		
	a	b	\$	S	S	A
I_0	SI_3	SI_4	—	—	I_1	I_2
I_1	—	—	Accept	—	—	—
I_2	SI_3	SI_4	—	—	—	I_5
I_3	SI_3	SI_4	—	—	—	I_6
I_4	r_4	r_4	r_4	—	—	—
I_5	r_2	r_2	(r_2)	—	—	—
I_6	r_3	r_3	r_3	—	—	—

Symbol table (do on your own)

a | b | a | b
↑

Stack

input

Action

\$	abab\$	Push I_0
\$ I_0	qbab\$	Shift I_3
\$ I_0 I_3	bab\$	Shift I_4
\$ I_0 I_3 I_4	Ab\$	reduce $A \rightarrow b$
\$ I_0 I_3 A I_6	ab\$	reduce reduce $A \rightarrow ab$
\$ I_0 A I_2	ab\$	Shift I_2
\$ I_0 A I_2 a I_3	b\$	Shift I_4
\$ I_0 A I_2 a I_3 b I_4	\$	reduce $A \rightarrow b$
\$ I_0 A I_2 A I_6	\$	reduce $A \rightarrow ab$
\$ I_0 A I_2 A I_5	\$	reduce $B \rightarrow AD$
\$ I_0 A I_2	\$	Accept

$$\frac{19}{20} \times 100$$

if action = shift

- Shift terminal
- Shift state

if action = reduce.

- Pop $2 \times | \beta |$
- Push variable A, $A \rightarrow \beta$
- Push goto [top of stack state A]

if action = accept.

else

error();

LR(0) parser has conflicts.

SR Conflict RR Conflict

$A \rightarrow b.$

Follow(A) $\rightarrow$ Put reduce here only

SLR(1)

5 $\rightarrow$ AA.
Follow(S) = \$

Implement LR(0) & SLR parser for given sub. of grammar

$E \rightarrow T + E$
 $E \rightarrow T$
 $T \rightarrow id$

(i + i)

$S \rightarrow aABC$
 $A \rightarrow Abc$
 $A \rightarrow b$
 $B \rightarrow d.$

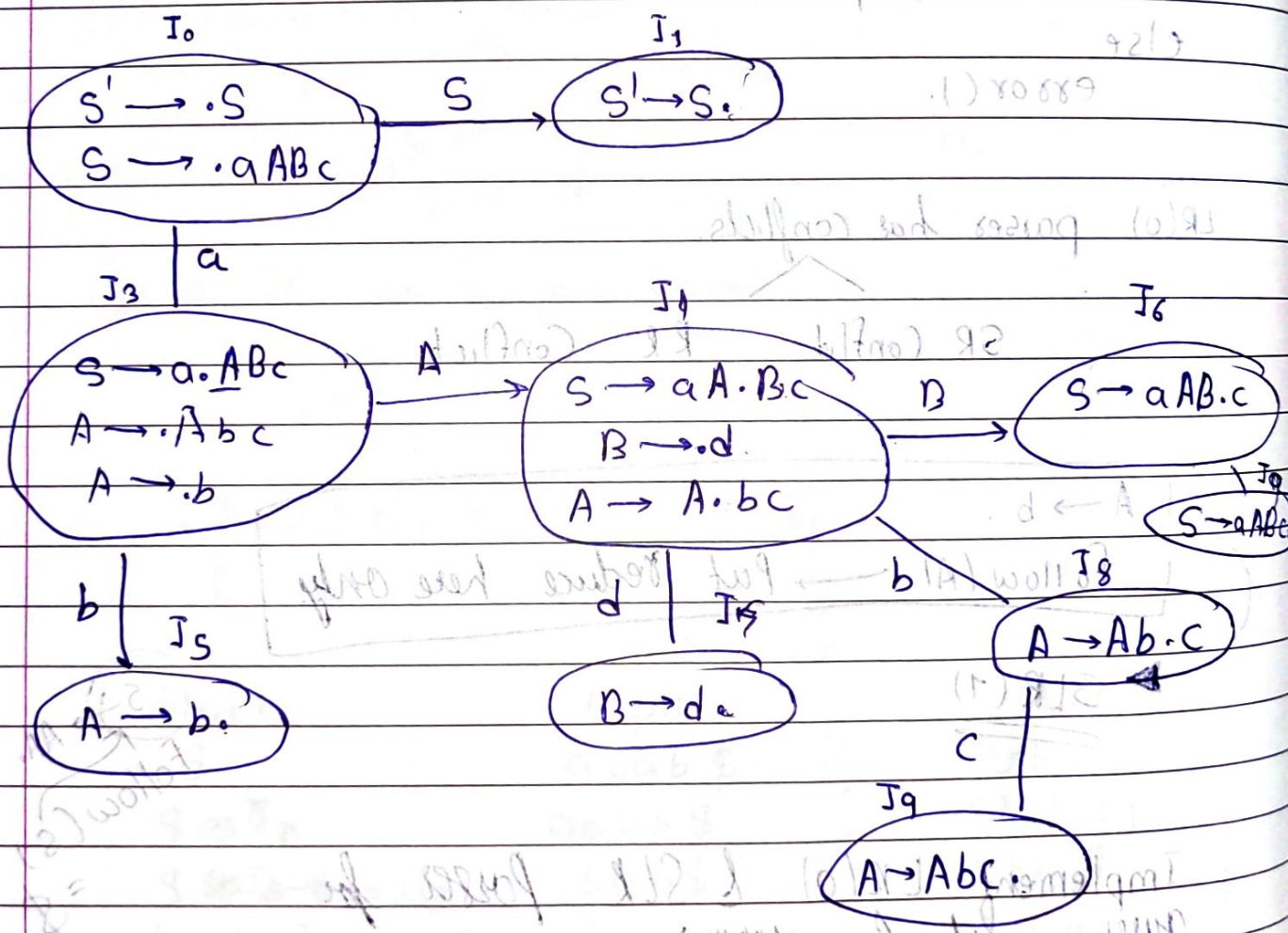
abbcddc

$$\begin{aligned} A &\rightarrow aA \\ A &\rightarrow b \\ A &\rightarrow \end{aligned}$$

$$\begin{aligned} A &\rightarrow aA|b|c \\ A' &\rightarrow A \\ A &\rightarrow \end{aligned}$$

1. Augmenting

$$\begin{aligned} S' &\rightarrow S & \text{--- ①} \\ S &\rightarrow aABc & \text{--- ②} \\ A &\rightarrow Abc & \text{--- ③} \\ A &\rightarrow b & \text{--- ④} \\ B &\rightarrow d & \text{--- ⑤} \end{aligned}$$



	a	b	d	\$	S	A	B
I_0	$S I_3$						
I_1							
I_2							
I_3							
I_4							

Bottom up parsing $\rightarrow$ SR $\rightarrow$ LR (All), Operator precedence parser, Symbol table management

CLR Parser

Canonical LR Parser

It uses LR(1) items

LR(1) items

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

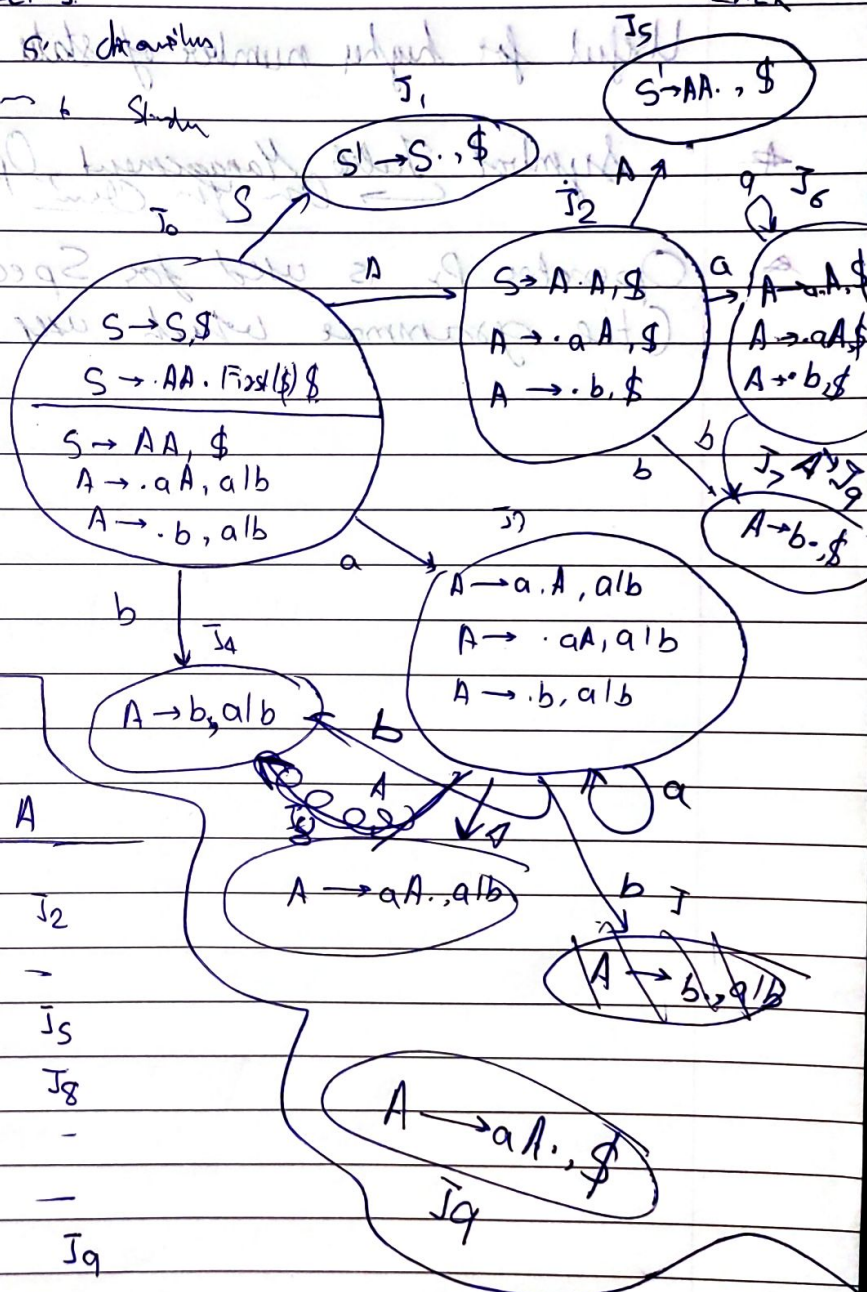
$\text{First}(A\$)$

$a, b, \$$

$= F_1(A)$

$= \{a, b\}$

	a	b	\$	S	A
I_0	$S I_3$	$S I_4$	$S I_5$	I_1	I_2
I_1	-	-	$A c a b$	-	-
I_2	$S I_3$	$S I_4$	-	-	I_5
I_3	$S I_3$	$S I_4$	-	-	I_8
I_4	γ_4	γ_4	-	-	-
I_5	-	-	γ_1	-	-
I_6	$S I_6$	$S I_7$	-	-	I_9
I_7	-	-	γ_4	-	-
I_8	γ_3	γ_3	-	-	-
I_9	-	-	γ_3	-	-



CLR is most powerful but more complex.